Version 3

Repetability Tests Analysis

Repitability test

- Rail length :110 cm endstop: 11.9 mm
- Piston stroke: 13.5 cm endstop: 8.4 mm

In [1]:
```
# Import Modules
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

# Read the CSV file into a DataFrame
data = pd.read_csv('DataAnalysisRepeatability.csv')
data.head(10)
```

Out[1]:

	Test Number	TEST	X_Point1	Y_Point1	Z_Point1	X_Point2	Y_Point2	Z_Point2	X_Poin
0	Test1	Piston	-213.276	-67.295	-32.011	-65.743	-46.577	-59.546	-139.4
1	Test2	Piston	-213.280	-67.298	-32.008	-65.749	-46.577	-59.541	-139.4
2	Test3	Piston	-213.283	-67.300	-32.003	-65.751	-46.580	-59.537	-139.4
3	Test4	Piston	-213.284	-67.300	-32.000	-65.750	-46.578	-59.535	-139.4
4	Test5	Piston	-213.284	-67.302	-31.997	-65.752	-46.581	-59.531	-139.4
5	Test6	Piston	-213.287	-67.305	-31.992	-65.754	-46.581	-59.524	-139.4
6	Test7	Piston	-213.287	-67.305	-31.994	-65.754	-46.581	-59.525	-139.4
7	Test8	Piston	-213.281	-67.313	-31.988	-65.745	-46.594	-59.518	-139.4
8	Test9	Piston	-213.281	-67.313	-31.988	-65.747	-46.591	-59.520	-139.4
9	Test10	Piston	-213.282	-67.312	-31.987	-65.747	-46.591	-59.518	-139.4

In [2]:
```
data.columns
```

Out[2]:
```
Index(['Test Number', 'TEST', 'X_Point1', 'Y_Point1', 'Z_Point1', 'X_Point2',
       'Y_Point2', 'Z_Point2', 'X_Point3', 'Y_Point3', 'Z_Point3'],
      dtype='object')
```

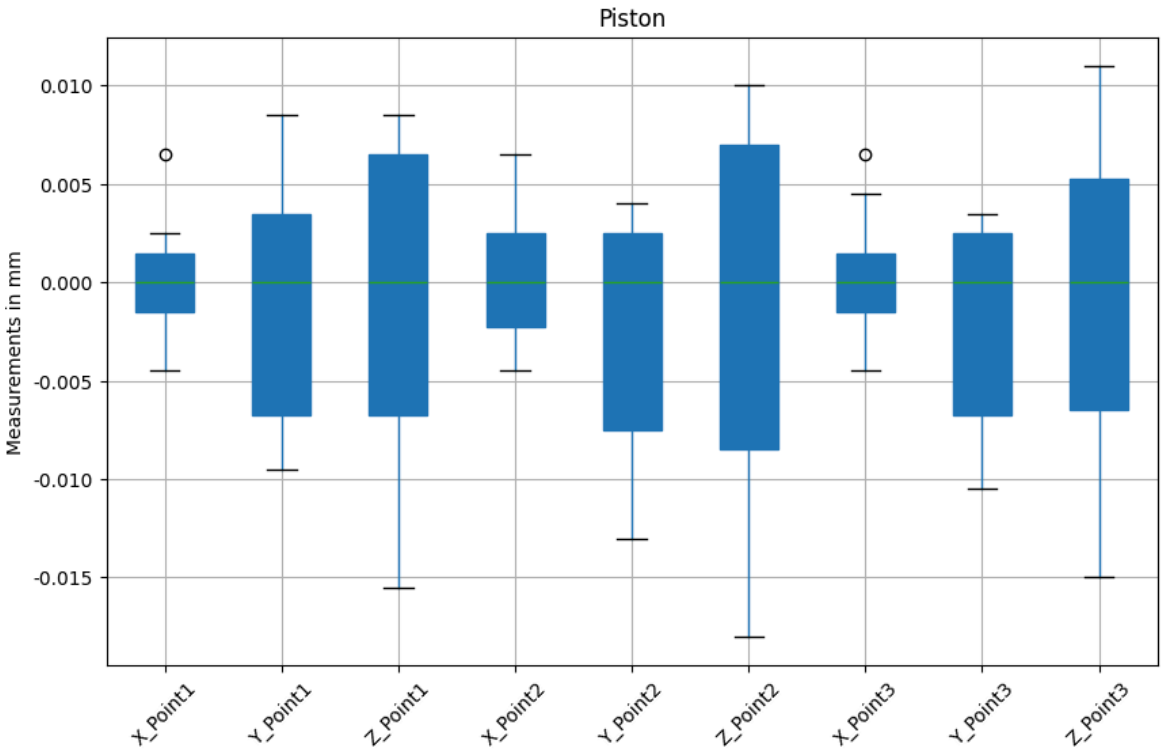
In [3]:
```
# Convert the column that contains 'TEST' in its name to categorical
data["TEST"] = pd.Categorical(data["TEST"])

# Looking into different types of Catagory
Category = data['TEST'].cat.categories
```

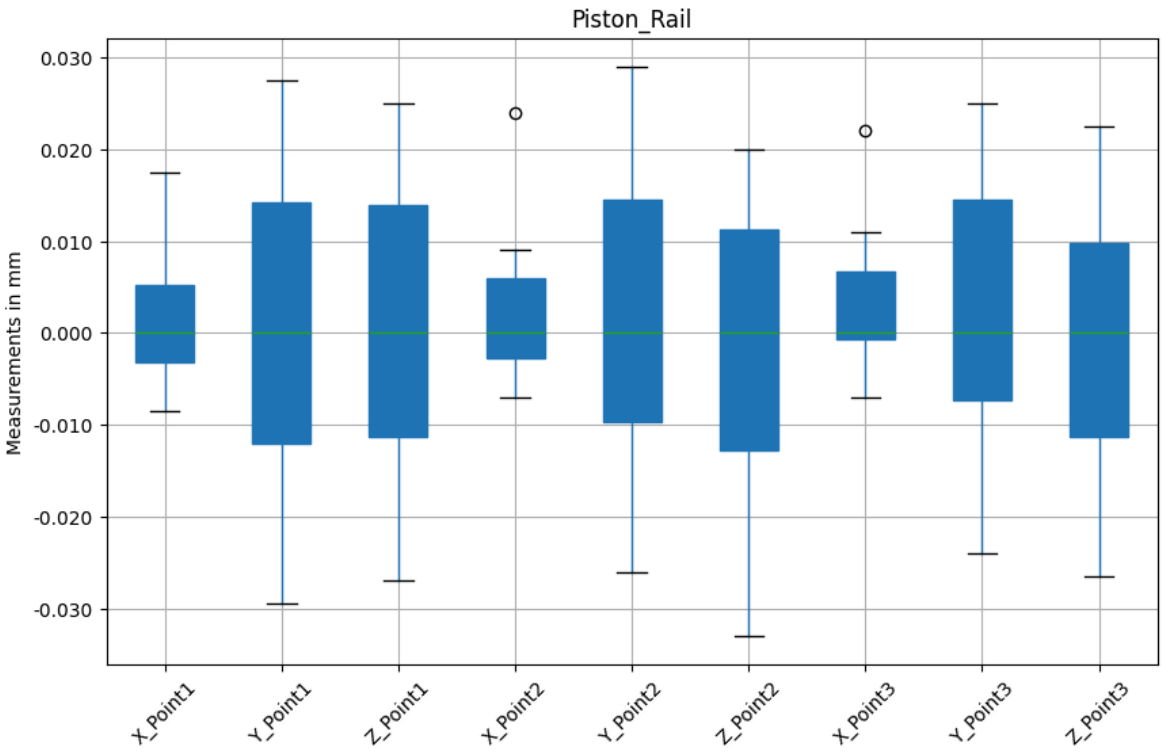
```
Category
#data.dtypes
```

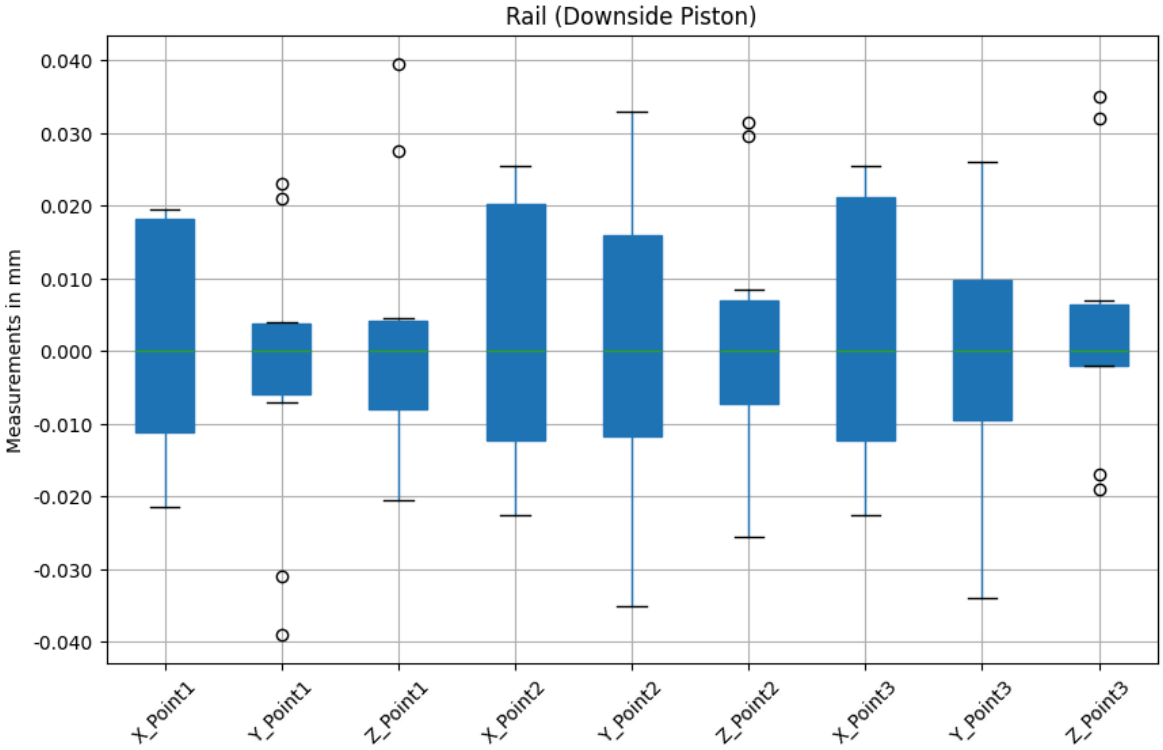
Out[3]: Index(['Piston', 'Piston_Rail', 'Rail (Downside Piston)'], dtype='object')

Box Plots

In [4]:
```
dict = {}

for i, different_tests in enumerate(Category):
    Key = Category[i]
    dict[Key] = data[data["TEST"] == Category[i]]

for Keys, Values in dict.items():
    # Identify categorical columns
    categorical_columns = Values.select_dtypes(include=['object', 'category']).c

    # Drop categorical columns
    no_categorical = Values.drop(columns=categorical_columns)

    #Standardize data for comperasion
    data_Standardize = no_categorical - no_categorical.median(axis=0)



    # Box Plot
    plt.figure(figsize=(10,6))
    data_Standardize.boxplot(patch_artist=True)
    plt.title(Keys)
    plt.ylabel("Measurements in mm")
    plt.xticks(rotation=45)
    plt.grid(True)
    plt.gca().yaxis.set_major_formatter(ticker.FormatStrFormatter('%.3f'))
    plt.show()
```



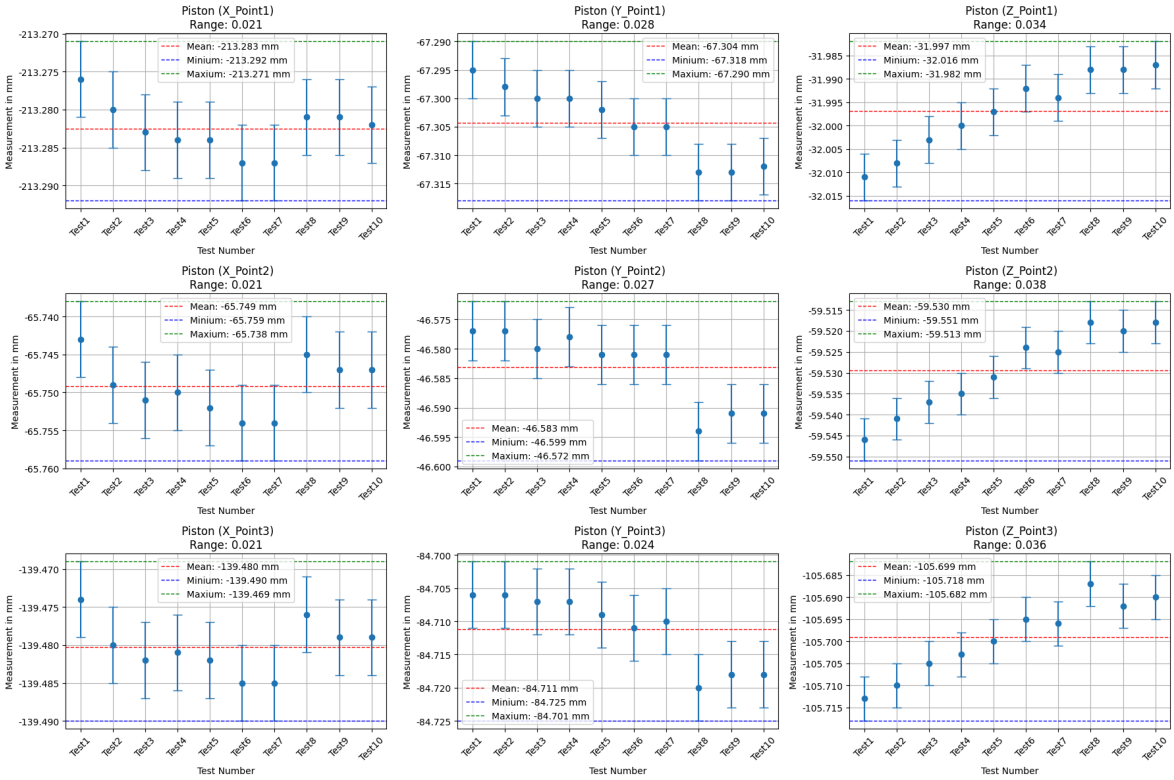


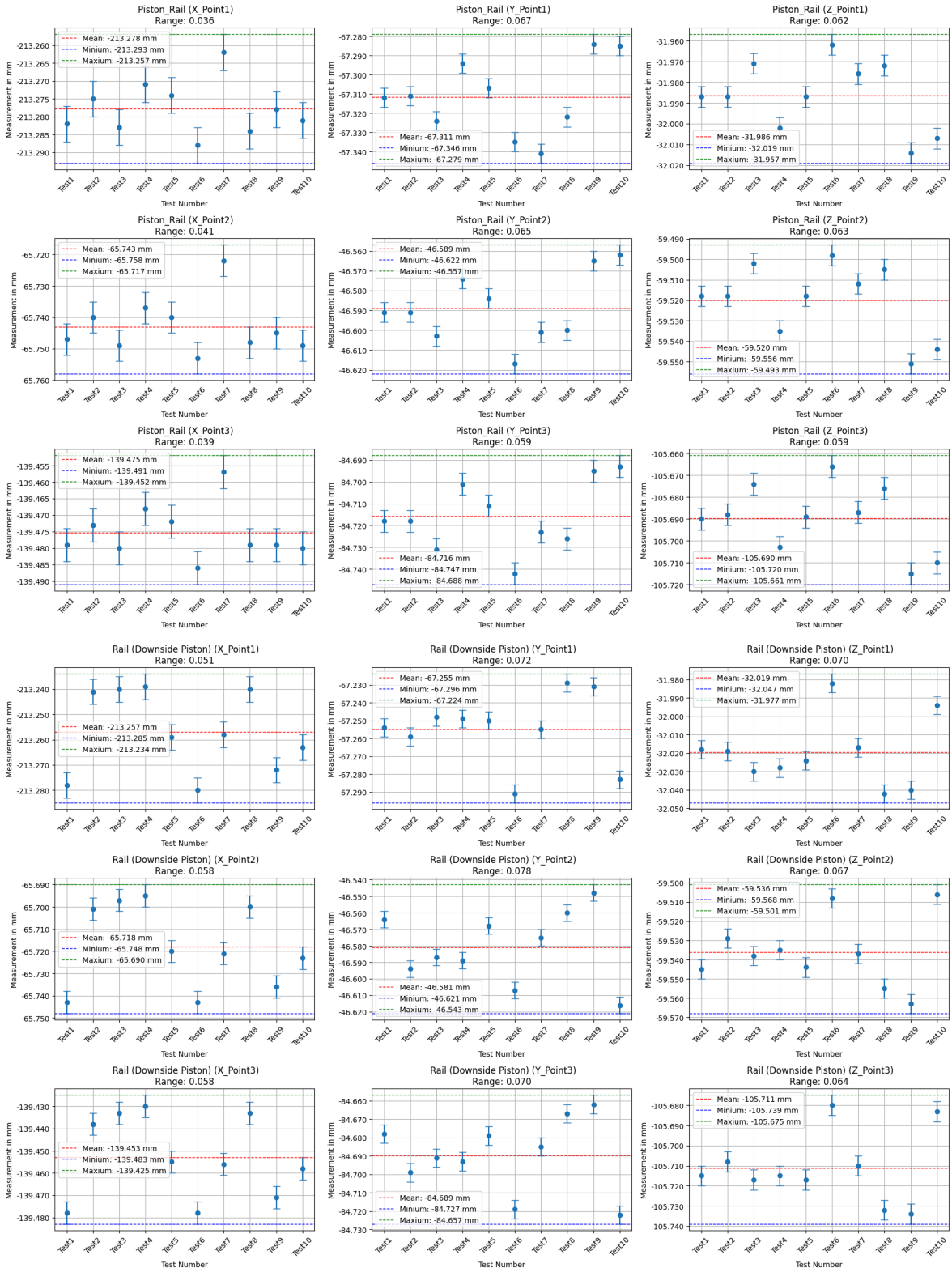


Error Bar

In [5]:
```
# Plot
machine_error = 5/1000 # in mm

mean_Range = {}

for Keys, Values in dict.items():
    # Create a figure with 6 subplots (3 rows, 3 columns)
    fig, axes = plt.subplots(3, 3, figsize=(18, 12))

    # Loop to generate six different graphs
```

```python
    for i, ax in enumerate(axes.flat, start=2):

        # Calculate the mean of the current column
        mean_value = dict[Keys][dict[Keys].columns[i]].mean()
        minium_Value = dict[Keys][dict[Keys].columns[i]].min()-machine_error
        maxium_Value = dict[Keys][dict[Keys].columns[i]].max()+ machine_error
        range = (maxium_Value - minium_Value)
        ax.errorbar(dict[Keys]["Test Number"], dict[Keys][dict[Keys].columns[i]]
        # Add a horizontal and Vertical mean line
        ax.axhline(y=mean_value, color='r', linestyle='--', linewidth=1, label=f
        ax.axhline(y=minium_Value, color='b', linestyle='--', linewidth=1, label
        ax.axhline(y=maxium_Value, color='g', linestyle='--', linewidth=1, label
        ax.set_title(f' {Keys} ({dict[Keys].columns[i]})\n Range: {range:.3f} ')
        ax.set_ylabel('Measurement in mm')
        ax.set_xlabel('Test Number')
        ax.tick_params(axis='x', labelrotation=45)
        ax.legend()
        ax.grid(True)
        ax.yaxis.set_major_formatter(ticker.FormatStrFormatter('%.3f'))
        #print(f' {Keys} ({dict[Keys].columns[i]}), Mean: {mean_value:.3f},  Err
        key = f' {Keys} ({dict[Keys].columns[i]})'
        mean_Range[key] = [mean_value, range]


    # Adjust layout to prevent overlap
    plt.tight_layout()
    plt.show()
```





Summary and output results to To CSV

In [6]:
```
outcomes = pd.DataFrame(mean_Range)
outcomes.index = ["Mean", "Range"]
outcomes = outcomes.T.round(3)
outcomes.to_csv("Outcomes.csv")
outcomes.head()
```

Out[6]:

	Mean	Range
Piston (X_Point1)	-213.283	0.021
Piston (Y_Point1)	-67.304	0.028
Piston (Z_Point1)	-31.997	0.034
Piston (X_Point2)	-65.749	0.021
Piston (Y_Point2)	-46.583	0.027

In [7]:
```
print("Maximum Error:", max(outcomes["Range"]))
```

```
Maximum Error: 0.078
```